KV32 SoC SDK API Reference

KV32 SoC Project

2026-02-28

Table of Contents

1. Abstract . . . 1
2. Introduction . . . 2
   2.1. Document Scope . . . 2
   2.2. SDK Architecture . . . 2
   2.3. Header Organization . . . 2
3. System Memory Map . . . 4
   3.1. Address Space Overview . . . 4
   3.2. Memory-Mapped Register Access . . . 4
4. Capability Register Definitions (`kv_cap.h`) . . . 5
   4.1. Overview . . . 5
   4.2. Capability Register Layout . . . 5
   4.3. Constants . . . 5
5. UART Driver API . . . 7
   5.1. Overview . . . 7
   5.2. Register Map . . . 7
   5.3. Status Bits . . . 7
   5.4. API Functions . . . 7
      5.4.1. Initialization . . . 7
      5.4.2. Status Queries . . . 8
      5.4.3. Polling Transmit . . . 8
      5.4.4. Polling Receive . . . 8
      5.4.5. Interrupt Control . . . 9
      5.4.6. Loopback Mode . . . 9
      5.4.7. Capability Register . . . 9
6. SPI Driver API . . . 11
   6.1. Overview . . . 11
   6.2. Register Map . . . 11
   6.3. SPI Modes . . . 11
   6.4. API Functions . . . 12
      6.4.1. Initialization . . . 12
      6.4.2. Chip Select Control . . . 12
      6.4.3. Data Transfer . . . 12
      6.4.4. Status and Control . . . 13
      6.4.5. Interrupt Control . . . 13
      6.4.6. Loopback Mode . . . 13
      6.4.7. Capability Register . . . 13
7. I2C Driver API . . . 15
   7.1. Overview . . . 15

7.2. Register Map . . . 15
7.3. API Functions . . . 15
7.3.1. Initialization . . . 15
7.3.2. Low-Level Operations . . . 16
7.3.3. High-Level Transfers . . . 16
7.3.4. Status and Control . . . 17
7.3.5. Interrupt Control . . . 17
7.3.6. Capability Register . . . 17
8. DMA Controller API . . . 19
8.1. Overview . . . 19
8.2. Register Map . . . 19
8.2.1. Per-Channel Registers . . . 19
8.2.2. Global Registers . . . 20
8.3. Control Register Bits . . . 20
8.4. Status Register Bits . . . 20
8.5. Transfer Modes . . . 20
8.5.1. 1D Transfer Mode . . . 20
8.5.2. 2D Strided Transfer Mode . . . 21
8.5.3. 3D Planar Transfer Mode . . . 21
8.5.4. Scatter-Gather Transfer Mode . . . 22
8.6. SDK API Functions . . . 22
8.6.1. Channel Lifecycle . . . 22
8.6.2. High-Level Transfer Functions . . . 22
8.6.3. Interrupt Functions . . . 23
8.6.4. Performance Counter Functions . . . 23
8.6.5. Identification and Capability . . . 23
8.7. Performance Monitoring . . . 24
9. Timer Driver API . . . 25
9.1. Overview . . . 25
9.2. Register Map . . . 25
9.3. API Functions . . . 26
9.3.1. Initialization . . . 26
9.3.2. Timer Mode . . . 26
9.3.3. PWM Mode . . . 26
9.3.4. Interrupt Handling . . . 26
9.3.5. Capability Register . . . 26
9.4. Usage Example . . . 27
10. GPIO Driver API . . . 28
10.1. Overview . . . 28
10.2. Register Map . . . 28
10.3. API Functions . . . 29

10.3.1. Initialization . . . 29
10.3.2. Direction and Output . . . 29
10.3.3. Input . . . 29
10.3.4. Interrupt Configuration . . . 29
10.3.5. Interrupt Status . . . 29
10.3.6. Loopback . . . 30
10.3.7. Capability Register . . . 30
11. Watchdog Timer (WDT) Driver API . . . 31
11.1. Overview . . . 31
11.2. Quick Reference . . . 31
11.3. Usage — IRQ Mode . . . 31
11.4. Usage — Reset Mode . . . 32
12. Platform-Level Interrupt Controller (PLIC) . . . 33
12.1. Overview . . . 33
12.2. Interrupt Sources . . . 33
12.3. API Functions . . . 33
12.3.1. Source Control . . . 33
12.3.2. Interrupt Handling . . . 34
12.3.3. Initialization Helper . . . 34
13. Core-Local Interrupt Controller (CLINT) . . . 35
13.1. Overview . . . 35
13.2. Features . . . 35
13.3. API Functions . . . 35
13.3.1. Timer Operations . . . 35
13.3.2. Software Interrupts . . . 36
14. Interrupt Management API . . . 37
14.1. Overview . . . 37
14.2. Interrupt Source Bits (mie/mip) . . . 37
14.3. Interrupt Cause Codes (mcause) . . . 37
14.4. Exception Cause Codes . . . 37
14.5. API Functions . . . 37
14.5.1. Global Interrupt Control . . . 37
14.5.2. Per-Source Control . . . 38
14.5.3. Wait For Interrupt . . . 38
14.5.4. Handler Registration . . . 38
15. Exception Handling and Trap Frame . . . 40
15.1. Overview . . . 40
15.2. Trap Frame Layout (`kv_trap_frame_t`) . . . 40
15.3. Design Rationale — Frame Pointer Instead of Direct CSR Writes . . . 41
15.3.1. Why the Old Approach Was Fragile . . . 41
15.3.2. The Fix: Frame Is the Authoritative Source . . . 41

15.3.3. Additional Benefits . . . 42
15.3.4. Interaction with the RTOS Port . . . 42
15.4. Writing Exception Handlers . . . 42
15.4.1. Skip a Faulting Instruction . . . 42
15.4.2. Redirect to a Pre-Stored Resume Address . . . 43
15.4.3. Emulate an ecall (Syscall ABI) . . . 43
15.5. Writing Interrupt Handlers . . . 43
15.6. Overriding `trap_handler` Entirely . . . 44
15.7. Trap Entry / Exit Flow . . . 44
16. Control and Status Registers (CSR) . . . 46
16.1. Overview . . . 46
16.2. Machine Information Registers . . . 46
16.3. Machine Trap Setup . . . 46
16.4. Machine Trap Handling . . . 46
16.5. Performance Counters . . . 47
17. Magic Device (Simulation/Test) . . . 48
17.1. Overview . . . 48
17.2. Registers . . . 48
17.3. AXI Decode and Responses . . . 48
17.4. Exit Code Encoding . . . 48
17.5. API Functions . . . 49
18. Complete Examples . . . 50
18.1. Example 1: UART Echo with Interrupts . . . 50
18.2. Example 2: SPI Flash Read . . . 51
18.3. Example 3: I2C EEPROM Access . . . 51
18.4. Example 4: DMA Memory Copy with Performance Monitoring . . . 53
19. Build System Integration . . . 55
19.1. Makefile Configuration . . . 55
19.2. Linker Script . . . 55
19.3. Startup Code . . . 55
19.4. Common Runtime . . . 55
20. Appendix A: Register Quick Reference . . . 56
20.1. UART Registers . . . 56
20.2. SPI Registers . . . 56
20.3. I2C Registers . . . 56
20.4. DMA Channel Registers . . . 57
20.5. Timer Registers . . . 57
20.6. GPIO Registers . . . 58
21. Appendix B: Interrupt Vector Table . . . 59
22. Appendix C: Performance Considerations . . . 60
22.1. DMA vs CPU Copy . . . 60

22.2. Cache Coherency . . . 60
22.3. Interrupt Latency . . . 60
23. Appendix D: Debugging Tips . . . 62
23.1. Enable Instruction Trace . . . 62
23.2. Generate Waveforms . . . 62
23.3. Common Issues . . . 62
24. Appendix E: RISC-V Compliance and KV32 Extensions . . . 63
24.1. Scope . . . 63
24.2. Classification Matrix . . . 63
24.3. Important KV32 Notes . . . 64
24.3.1. `fence.i` behavior . . . 64
24.3.2. PMA CSRs are custom . . . 64
24.3.3. Trap redirection model . . . 64
24.4. Portability Checklist . . . 64
Document Information . . . 65

Chapter 1. Abstract

This document provides a comprehensive reference for the KV32 SoC Software Development Kit (SDK). It describes the API functions, register definitions, and programming model for all peripherals and system components in the KV32 RISC-V System-on-Chip.

The SDK provides a header-only C library optimized for bare-metal embedded applications, with zero overhead when functions are not used.

Chapter 2. Introduction

2.1. Document Scope

This API reference covers:

- Memory-mapped I/O (MMIO) peripheral drivers
- Interrupt controller configuration (PLIC, CLINT)
- Control and Status Register (CSR) access
- System initialization and exception handling
- Debug and simulation utilities

2.2. SDK Architecture

The KV32 SDK follows these design principles:

Header-Only Design

All driver functions are implemented as `static inline` functions, eliminating the need to link against a separate library. Unused functions contribute zero code size.

Zero Overhead Abstraction

Direct register access with no additional memory or CPU overhead compared to raw MMIO operations.

Type Safety

All register addresses and bit fields are defined as compile-time constants with proper types.

Portability

Works with both RTL simulation and software simulators (kv32sim, Spike).

2.3. Header Organization

Header File	Peripheral/Module	Description
`kv_platform.h`	Platform definitions	Base addresses, register offsets, bit-field constants for all peripherals
`kv_uart.h`	UART	Serial communication driver (polling and interrupt modes)
`kv_spi.h`	SPI	SPI master controller driver
`kv_i2c.h`	I2C	I2C master controller driver
`kv_plic.h`	PLIC	Platform-Level Interrupt Controller configuration

Header File	Peripheral/Module	Description
`kv_clint.h`	CLINT	Core-Local Interrupt Controller (timer and software interrupts)
`kv_irq.h`	Interrupt Management	Global and per-source interrupt control, handler registration
`csr.h`	CSR Access	Control and Status Register read/write functions
`kv_dma.h`	DMA Controller	DMA controller driver (channel management, 1D/2D/3D/SG transfers, IRQ, performance)
`kv_timer.h`	Timer/PWM	Timer/PWM driver (polling and interrupt modes)
`kv_gpio.h`	GPIO	General-Purpose I/O driver
`kv_wdt.h`	Watchdog Timer	Hardware watchdog timer driver (start, kick, stop, IRQ clear)
`kv_cap.h`	Capability Constants	Expected capability register values for all peripherals (derived from RTL parameters)

Chapter 3. System Memory Map

3.1. Address Space Overview

Region	Base Address	Size	Description
Boot ROM	`0x0000_0000`	64 KB	Read-only boot code (synthesized firmware)
Main RAM	`0x8000_0000`	2 MB	Program and data memory (SRAM)
CLINT	`0x0200_0000`	64 KB	Core-Local Interrupt Controller
PLIC	`0x0C00_0000`	64 MB	Platform-Level Interrupt Controller
UART	`0x2001_0000`	64 KB	Serial communication peripheral
I2C	`0x2002_0000`	64 KB	I2C master controller
SPI	`0x2003_0000`	64 KB	SPI master controller
DMA	`0x2000_0000`	64 KB	Direct Memory Access controller (8 channels)
Magic Device	`0x4000_0000`	64 KB	Simulation/testbench console and exit control

3.2. Memory-Mapped Register Access

All peripherals use 32-bit aligned registers. The SDK provides the `KV_REG32` macro for safe MMIO access:

```
#define KV_REG32(base, off) \
    (*(volatile uint32_t *)((uintptr_t)(base) + (uintptr_t)(off)))
```

Example usage:

```
uint32_t status = KV_REG32(KV_UART_BASE, KV_UART_STATUS_OFF);
KV_REG32(KV_UART_BASE, KV_UART_DATA_OFF) = 0x41; // Write 'A'
```

Chapter 4. Capability Register Definitions (kv_cap.h)

4.1. Overview

The kv_cap.h header provides compile-time constants that mirror the RTL CAPABILITY_REG localparams for each peripheral. These values can be compared at runtime against the hardware capability register to verify RTL and SDK are in sync.

Include this header in any file that reads capability registers:

```
#include "kv_cap.h"
```

4.2. Capability Register Layout

Peripheral	Bit Layout	Expected Value
UART	[31:16] VERSION, [15:8] RX_FIFO_DEPTH, [7:0] TX_FIFO_DEPTH	0x00011010 (v0.0001, FIFO=16)
I2C	[31:16] VERSION, [15:8] RX_FIFO_DEPTH, [7:0] TX_FIFO_DEPTH	0x00010808 (v0.0001, FIFO=8)
SPI	[31:16] VERSION, [23:20] NUM_CS, [19:16] RX_FIFO(4-bit), [7:0] TX_FIFO_DEPTH	0x00014808 (v0.0001, 4 CS, FIFO=8)
DMA	[31:16] VERSION, [15:8] NUM_CHANNELS, [7:0] MAX_BURST_LEN	0x00010410 (v0.0001, 4 ch, burst=16)
GPIO	[31:16] VERSION, [15:8] NUM_BANKS, [7:0] NUM_PINS	0x00010104 (v0.0001, 1 bank, 4 pins)
Timer	[31:16] VERSION, [15:8] NUM_CHANNELS, [7:0] COUNTER_WIDTH	0x00010420 (v0.0001, 4 ch, 32-bit)

4.3. Constants

```
/* UART */
#define KV_CAP_UART_VERSION        0x0001u
#define KV_CAP_UART_TX_FIFO_DEPTH  16u
```

```
#define KV_CAP_UART_RX_FIFO_DEPTH  16u
#define KV_CAP_UART_VALUE          0x00011010u

/* I2C */
#define KV_CAP_I2C_VERSION         0x0001u
#define KV_CAP_I2C_TX_FIFO_DEPTH   8u
#define KV_CAP_I2C_RX_FIFO_DEPTH   8u
#define KV_CAP_I2C_VALUE           0x00010808u

/* SPI */
#define KV_CAP_SPI_VERSION         0x0001u
#define KV_CAP_SPI_NUM_CS          4u
#define KV_CAP_SPI_TX_FIFO_DEPTH   8u
#define KV_CAP_SPI_RX_FIFO_DEPTH   8u
#define KV_CAP_SPI_VALUE           0x00014808u

/* DMA */
#define KV_CAP_DMA_VERSION         0x0001u
#define KV_CAP_DMA_NUM_CHANNELS    4u
#define KV_CAP_DMA_MAX_BURST_LEN   16u
#define KV_CAP_DMA_VALUE           0x00010410u

/* GPIO */
#define KV_CAP_GPIO_VERSION        0x0001u
#define KV_CAP_GPIO_NUM_PINS       4u
#define KV_CAP_GPIO_NUM_BANKS      1u
#define KV_CAP_GPIO_VALUE          0x00010104u

/* Timer */
#define KV_CAP_TIMER_VERSION        0x0001u
#define KV_CAP_TIMER_NUM_CHANNELS   4u
#define KV_CAP_TIMER_COUNTER_WIDTH  32u
#define KV_CAP_TIMER_VALUE          0x00010420u
```

Chapter 5. UART Driver API

5.1. Overview

The UART peripheral (`axi_uart.sv`) provides full-duplex serial communication with:

- Configurable baud rate
- 8-bit data, no parity, 1 stop bit (8N1 format)
- TX and RX FIFOs (16 entries each)
- Interrupt generation on RX data available and TX FIFO empty
- Internal loopback mode for testing

Base Address

`0x2001_0000` (KV_UART_BASE)

PLIC Source ID

1 (KV_PLIC_SRC_UART)

5.2. Register Map

Offset	Name	Access	Description
`0x00`	DATA	R/W	TX (write) / RX (read) data register
`0x04`	STATUS	RO	Status flags (TX busy, RX ready, FIFO full)
`0x08`	IE	R/W	Interrupt Enable register
`0x0C`	IS	R/W1C	Interrupt Status register (write-1-to-clear)
`0x10`	LEVEL	R/W	Baud rate divisor
`0x14`	CTRL	R/W	Control register (loopback enable)

5.3. Status Bits

Bit	Name	Description
0	TX_BUSY / TX_FULL	TX FIFO full – cannot write
2	RX_READY	RX FIFO not empty – data available
3	RX_FULL	RX FIFO full

5.4. API Functions

5.4.1. Initialization

```
void kv_uart_init(uint32_t baud_div);
```

Initialize the UART peripheral with the specified baud rate divisor.

Parameters:

- `baud_div`: Baud rate divisor = `(sys_clk_hz / (baud_rate * 8)) - 1`

Example:

```
// Configure for 115200 baud at 100 MHz system clock
// baud_div = (100000000 / (115200 * 8)) - 1 = 107
kv_uart_init(107);
```

5.4.2. Status Queries

```
int kv_uart_tx_busy(void);    // Returns 1 if TX FIFO full
int kv_uart_rx_ready(void);   // Returns 1 if RX data available
```

5.4.3. Polling Transmit

```
void kv_uart_putc(char c);
void kv_uart_puts(const char *s);
void kv_uart_write(const uint8_t *buf, uint32_t len);
```

Description:

- `kv_uart_putc()`: Block until TX FIFO has space, then transmit one character
- `kv_uart_puts()`: Transmit a null-terminated string
- `kv_uart_write()`: Transmit len bytes from buffer

Example:

```
kv_uart_puts("Hello, World!\n");
```

5.4.4. Polling Receive

```
int kv_uart_getc(void);              // Non-blocking, returns -1 if empty
uint8_t kv_uart_getc_blocking(void); // Blocks until data available
```

Example:

```
int c;
while ((c = kv_uart_getc()) != -1) {
    kv_uart_putc((char)c);  // Echo received character
}
```

5.4.5. Interrupt Control

```
void kv_uart_irq_enable(uint32_t mask);
void kv_uart_irq_disable(uint32_t mask);
uint32_t kv_uart_irq_status(void);
```

Interrupt Sources:

- `KV_UART_IE_RX_READY` (bit 0): RX FIFO not empty
- `KV_UART_IE_TX_EMPTY` (bit 1): TX FIFO drained (empty)

Example:

```
// Enable RX interrupt
kv_uart_irq_enable(KV_UART_IE_RX_READY);

// In interrupt handler:
uint32_t status = kv_uart_irq_status();
if (status & KV_UART_IE_RX_READY) {
    char c = kv_uart_getc();
    // Process received character
}
```

5.4.6. Loopback Mode

```
void kv_uart_loopback_enable(void);
void kv_uart_loopback_disable(void);
```

Enable internal TX→RX loopback for hardware testing. Transmitted bytes are fed back to the RX path without leaving the chip.

5.4.7. Capability Register

```
uint32_t kv_uart_get_capability(void);    // Full 32-bit capability register
uint32_t kv_uart_get_tx_fifo_depth(void); // TX FIFO depth in bytes
uint32_t kv_uart_get_rx_fifo_depth(void); // RX FIFO depth in bytes
uint32_t kv_uart_get_version(void);       // Hardware version [31:16]
```

Reads the read-only capability register at offset `KV_UART_CAP_OFF` (`0x18`). Compare against `KV_CAP_UART_VALUE` from `kv_cap.h` to verify hardware matches SDK.

```
#include "kv_cap.h"
uint32_t cap = kv_uart_get_capability();
printf("UART cap: 0x%08X (expected 0x%08X)\n", cap, KV_CAP_UART_VALUE);
printf("TX FIFO: %u  RX FIFO: %u\n",
       kv_uart_get_tx_fifo_depth(), kv_uart_get_rx_fifo_depth());
```

Chapter 6. SPI Driver API

6.1. Overview

The SPI peripheral (`axi_spi.sv`) provides a master-mode SPI controller with:

- Configurable clock frequency
- All four SPI modes (CPOL, CPHA)
- Up to 4 independent chip-select lines
- TX and RX FIFOs (16 entries each)
- Full-duplex operation
- Internal loopback mode

Base Address

`0x2003_0000` (KV_SPI_BASE)

PLIC Source ID

2 (KV_PLIC_SRC_SPI)

6.2. Register Map

Offset	Name	Access	Description
`0x00`	CTRL	R/W	Control register (enable, CPOL, CPHA, CS, loopback)
`0x04`	DIV	R/W	Clock divisor
`0x08`	TX	WO	TX FIFO push
`0x0C`	RX	RO	RX FIFO pop
`0x10`	STATUS	RO	Status flags
`0x14`	IE	R/W	Interrupt Enable
`0x18`	IS	R/W1C	Interrupt Status (`IS[2]` sticky W1C; `IS[1:0]` level)

6.3. SPI Modes

Mode	CPOL	CPHA	Description
MODE0	0	0	Clock idle low, sample on rising edge
MODE1	0	1	Clock idle low, sample on falling edge
MODE2	1	0	Clock idle high, sample on falling edge
MODE3	1	1	Clock idle high, sample on rising edge

6.4. API Functions

6.4.1. Initialization

```
void kv_spi_init(uint32_t clk_div, uint32_t mode);
```

Parameters:

- `clk_div`: SCK frequency = `sys_clk / (2 * (clk_div + 1))`
- `mode`: `KV_SPI_MODE0`, `MODE1`, `MODE2`, or `MODE3`

Example:

```
// Configure for 1 MHz SPI clock at 100 MHz system clock, mode 0
// clk_div = (100000000 / (2 * 1000000)) - 1 = 49
kv_spi_init(49, KV_SPI_MODE0);
```

6.4.2. Chip Select Control

```
void kv_spi_cs_select(uint32_t n);   // Assert CS[n] (0-3)
void kv_spi_cs_deselect(void);       // Deassert all CS lines
```

Example:

```
kv_spi_cs_select(0);        // Assert CS0
// Perform transfer
kv_spi_cs_deselect();       // Release CS0
```

6.4.3. Data Transfer

```
uint8_t kv_spi_transfer(uint8_t data);
void kv_spi_transfer_buf(const uint8_t *tx_buf, uint8_t *rx_buf, uint32_t len);
void kv_spi_read_buf(uint8_t *rx_buf, uint32_t len);
```

Description:

- `kv_spi_transfer()`: Full-duplex transfer of one byte
- `kv_spi_transfer_buf()`: Transfer multiple bytes (tx_buf or rx_buf may be NULL)
- `kv_spi_read_buf()`: Read len bytes by clocking out 0xFF

Example:

```
uint8_t tx_data[] = {0x03, 0x00, 0x00, 0x00}; // Read command + address
uint8_t rx_data[256];

kv_spi_cs_select(0);
kv_spi_transfer_buf(tx_data, NULL, 4);  // Send command
kv_spi_read_buf(rx_data, 256);          // Read data
kv_spi_cs_deselect();
```

6.4.4. Status and Control

```
int kv_spi_busy(void);        // Transfer in progress
int kv_spi_tx_ready(void);    // Can accept TX data
int kv_spi_rx_valid(void);    // RX data available
void kv_spi_wait_ready(void); // Block until idle
```

6.4.5. Interrupt Control

```
void kv_spi_irq_enable(uint32_t mask);
void kv_spi_irq_disable(uint32_t mask);
uint32_t kv_spi_irq_status(void);
```

Interrupt Sources:

- `KV_SPI_IE_RX_READY` (bit 0): RX FIFO not empty
- `KV_SPI_IE_TX_EMPTY` (bit 1): TX FIFO empty

6.4.6. Loopback Mode

```
void kv_spi_loopback_enable(void);
void kv_spi_loopback_disable(void);
```

6.4.7. Capability Register

```
uint32_t kv_spi_get_capability(void);    // Full 32-bit capability register
uint32_t kv_spi_get_tx_fifo_depth(void); // TX FIFO depth
uint32_t kv_spi_get_rx_fifo_depth(void); // RX FIFO depth
uint32_t kv_spi_get_num_cs(void);        // Number of chip-select lines
uint32_t kv_spi_get_version(void);       // Hardware version [31:16]
```

Reads the read-only capability register at offset `KV_SPI_CAP_OFF` (`0x1C`). Compare against `KV_CAP_SPI_VALUE` from `kv_cap.h`.

```c
#include "kv_cap.h"
uint32_t cap = kv_spi_get_capability();
printf("SPI cap: 0x%08X  CS=%u TX_FIFO=%u\n",
       cap, kv_spi_get_num_cs(), kv_spi_get_tx_fifo_depth());
```

Chapter 7. I2C Driver API

7.1. Overview

The I2C peripheral (`axi_i2c.sv`) provides a master-mode I2C controller with:

- Configurable clock frequency (100 kHz / 400 kHz / custom)
- 7-bit addressing
- START/STOP condition generation
- ACK/NACK detection
- Interrupt support

Base Address

`0x2002_0000` (KV_I2C_BASE)

PLIC Source ID

3 (KV_PLIC_SRC_I2C)

7.2. Register Map

Offset	Name	Access	Description
`0x00`	CTRL	R/W	Control register (enable, START, STOP, READ, NACK)
`0x04`	DIV	R/W	Clock divisor
`0x08`	TX	WO	Transmit data register
`0x0C`	RX	RO	Receive data register
`0x10`	STATUS	RO	Status flags (busy, ACK received, RX valid)
`0x14`	IE	R/W	Interrupt Enable
`0x18`	IS	R/W1C	Interrupt Status

7.3. API Functions

7.3.1. Initialization

```
void kv_i2c_init(uint32_t clk_div);
```

Parameters:

- `clk_div`: SCL period = `sys_clk / (4 * (clk_div + 1))`

Example:

```
// Configure for 100 kHz at 100 MHz system clock
// clk_div = (100000000 / (4 * 100000)) - 1 = 249
kv_i2c_init(249);

// Configure for 400 kHz at 100 MHz system clock
// clk_div = (100000000 / (4 * 400000)) - 1 = 62
kv_i2c_init(62);
```

7.3.2. Low-Level Operations

```
void kv_i2c_start(void);                    // Issue START condition
void kv_i2c_stop(void);                     // Issue STOP condition
int kv_i2c_write_byte(uint8_t data);        // Write byte, return 0 if ACK
uint8_t kv_i2c_read_byte(int send_ack);     // Read byte, send ACK if 1
```

7.3.3. High-Level Transfers

```
int kv_i2c_master_write(uint8_t addr, const uint8_t *buf, uint32_t len);
int kv_i2c_master_read(uint8_t addr, uint8_t *buf, uint32_t len);
```

Description:

Complete write or read transactions including START, address, data, and STOP.

Parameters:

- `addr`: 7-bit I2C device address
- `buf`: Data buffer
- `len`: Number of bytes to transfer

Return Values:

- `0`: Success
- `-1`: Device did not ACK the address
- `-2`: Device did not ACK a data byte

Example:

```
// Write to EEPROM at address 0x50
uint8_t eeprom_addr = 0x50;
uint8_t write_data[] = {0x00, 0x10, 0xAA, 0xBB}; // Addr MSB, LSB, data

if (kv_i2c_master_write(eeprom_addr, write_data, 4) < 0) {
    // Handle error
}
```

```
// Read from EEPROM
uint8_t read_buf[2];
if (kv_i2c_master_read(eeprom_addr, read_buf, 2) < 0) {
    // Handle error
}
```

7.3.4. Status and Control

```
int kv_i2c_busy(void);           // Transfer in progress
int kv_i2c_ack_received(void);   // Slave ACKed last byte
int kv_i2c_rx_valid(void);       // RX data available
void kv_i2c_wait_ready(void);    // Block until idle
```

7.3.5. Interrupt Control

```
void kv_i2c_irq_enable(uint32_t mask);
void kv_i2c_irq_disable(uint32_t mask);
uint32_t kv_i2c_irq_status(void);
```

Interrupt Sources:

- `KV_I2C_IE_RX_READY` (bit 0): RX FIFO not empty
- `KV_I2C_IE_TX_EMPTY` (bit 1): TX FIFO empty
- `KV_I2C_IE_STOP_DONE` (bit 2): STOP condition completed (sticky until W1C clear)

`kv_i2c_irq_status()` reads `IS` and writes the same value back. This acknowledges `IS[2]` (W1C STOP_DONE); `IS[1:0]` are level-driven and clear when the underlying FIFO/busy condition deasserts.

7.3.6. Capability Register

```
uint32_t kv_i2c_get_capability(void);    // Full 32-bit capability register
uint32_t kv_i2c_get_tx_fifo_depth(void); // TX FIFO depth
uint32_t kv_i2c_get_rx_fifo_depth(void); // RX FIFO depth
uint32_t kv_i2c_get_version(void);       // Hardware version [31:16]
```

Reads the read-only capability register at offset `KV_I2C_CAP_OFF` (`0x1C`). Compare against `KV_CAP_I2C_VALUE` from `kv_cap.h`.

```
#include "kv_cap.h"
uint32_t cap = kv_i2c_get_capability();
printf("I2C cap: 0x%08X  TX_FIFO=%u  RX_FIFO=%u\n",
       cap, kv_i2c_get_tx_fifo_depth(), kv_i2c_get_rx_fifo_depth());
```

Chapter 8. DMA Controller API

8.1. Overview

The DMA controller (`axi_dma.sv`) provides high-performance data transfers with:

- 4 independent channels
- Multiple transfer modes: 1D, 2D strided, 3D planar, scatter-gather
- Configurable source/destination address increment
- Interrupt generation on completion or error
- Performance counters for throughput monitoring

Base Address

`0x2000_0000` (KV_DMA_BASE)

PLIC Source ID

4 (KV_PLIC_SRC_DMA)

8.2. Register Map

8.2.1. Per-Channel Registers

Each channel has registers at `KV_DMA_BASE + channel * 0x40`:

Offset	Name	Access	Description
`0x00`	CTRL	R/W	Control register
`0x04`	STAT	R/W1C	Status register
`0x08`	SRC	R/W	Source address
`0x0C`	DST	R/W	Destination address
`0x10`	XFER	R/W	Transfer size (bytes)
`0x14`	SSTRIDE	R/W	Source stride (2D mode)
`0x18`	DSTRIDE	R/W	Destination stride (2D mode)
`0x1C`	ROWCNT	R/W	Row count (2D/3D mode)
`0x20`	SPSTRIDE	R/W	Source plane stride (3D mode)
`0x24`	DPSTRIDE	R/W	Destination plane stride (3D mode)
`0x28`	PLANECNT	R/W	Plane count (3D mode)
`0x2C`	SGADDR	R/W	Scatter-gather descriptor address
`0x30`	SGCNT	R/W	Scatter-gather descriptor count

8.2.2. Global Registers

Offset	Name	Access	Description
`0xF00`	IRQ_STAT	R/W1C	Channel done/error flags (one bit per channel)
`0xF04`	IRQ_EN	R/W	Global interrupt enable per channel
`0xF08`	ID	RO	Identification register (0xD4A00100)
`0xF0C`	CAPABILITY	RO	Hardware capability: `{VERSION[31:16], NUM_CHANNELS[15:8], MAX_BURST_LEN[7:0]}`
`0xF10`	PERF_CTRL	R/W	Performance counter control
`0xF14`	PERF_CYCLES	R/W	Cycles elapsed
`0xF18`	PERF_RD_BYTES	R/W	Total bytes read
`0xF1C`	PERF_WR_BYTES	R/W	Total bytes written

8.3. Control Register Bits

Bit(s)	Name	Description
0	EN	Enable channel
1	START	Arm transfer (auto-clears)
2	STOP	Abort transfer (auto-clears)
4:3	MODE	`00`=1D, `01`=2D, `10`=3D, `11`=Scatter-Gather
5	SRC_INC	Increment source address
6	DST_INC	Increment destination address
7	IE	Interrupt enable for this channel

8.4. Status Register Bits

Bit	Name	Description
0	BUSY	Transfer in progress (read-only)
1	DONE	Transfer complete (write-1-to-clear)
2	ERR	AXI error occurred (write-1-to-clear)

8.5. Transfer Modes

8.5.1. 1D Transfer Mode

Simple linear memory copy using `kv_dma_1d_copy()`:

```c
#include "kv_dma.h"

// Copy 1024 bytes from src to dst on channel 0, then verify done
int ret = kv_dma_1d_copy(0, src_addr, dst_addr, 1024);
if (ret != 0) { /* AXI error */ }
kv_dma_ch_reset(0);  // Clear DONE/ERR flags
```

For non-blocking use, set up the channel manually and call `kv_dma_ch_start()`:

```c
kv_dma_ch_reset(0);
KV_DMA_CH_REG(0, KV_DMA_CH_SRC_OFF)  = src_addr;
KV_DMA_CH_REG(0, KV_DMA_CH_DST_OFF)  = dst_addr;
KV_DMA_CH_REG(0, KV_DMA_CH_XFER_OFF) = 1024;
KV_DMA_CH_REG(0, KV_DMA_CH_CTRL_OFF) = KV_DMA_CTRL_EN | KV_DMA_CTRL_MODE_1D
                                     | KV_DMA_CTRL_SRC_INC | KV_DMA_CTRL_DST_INC;
kv_dma_ch_start(0);
// ... do other work ...
kv_dma_ch_wait(0);   // Blocks until done or error
kv_dma_ch_reset(0);
```

8.5.2. 2D Strided Transfer Mode

Copy rectangular regions with different pitches using `kv_dma_2d_copy()`:

```c
// Copy 640x480 image; src stride=1024 bytes/row, dst stride=800 bytes/row
int ret = kv_dma_2d_copy(0,
    src_addr, dst_addr,
    640,   /* row_bytes  */
    480,   /* num_rows   */
    1024,  /* src_stride */
    800);  /* dst_stride */
kv_dma_ch_reset(0);
```

8.5.3. 3D Planar Transfer Mode

Copy 3D data structures (e.g., video frames) using `kv_dma_3d_copy()`:

```c
// Copy video buffer: 640x480x3 (RGB planes)
// Each plane: 640 bytes/row, 480 rows; planes packed contiguously
int ret = kv_dma_3d_copy(0,
    src_addr, dst_addr,
    640,    /* row_bytes    */
    480,    /* num_rows     */
    3,      /* num_planes   */
    640,    /* src_stride   */
    640,    /* dst_stride   */
```

```
    307200, /* src_pstride  */
    307200  /* dst_pstride  */);
kv_dma_ch_reset(0);
```

8.5.4. Scatter-Gather Transfer Mode

Process a descriptor list using `kv_dma_sg_copy()`:

```
typedef struct {
    uint32_t src_addr;
    uint32_t dst_addr;
    uint32_t xfer_cnt;
    uint32_t mode_ctrl;  /* [3]=dst_inc, [2]=src_inc, [1:0]=mode */
} sg_desc_t;

__attribute__((aligned(16))) sg_desc_t descs[3] = {
    { src + 0,  dst + 0,  512, 0x0C },
    { src + 512, dst + 512, 256, 0x0C },
    { src + 768, dst + 768, 256, 0x0C },
};

int ret = kv_dma_sg_copy(0, (uint32_t)(uintptr_t)descs, 3);
kv_dma_ch_reset(0);
```

8.6. SDK API Functions

8.6.1. Channel Lifecycle

```
void    kv_dma_ch_reset(int ch);    // Disable channel, clear DONE/ERR
void    kv_dma_ch_start(int ch);    // Set START bit (arms transfer)
void    kv_dma_ch_stop(int ch);     // Set STOP bit (aborts transfer)
int     kv_dma_ch_busy(int ch);     // Returns 1 if BUSY
int     kv_dma_ch_done(int ch);     // Returns 1 if DONE
int     kv_dma_ch_error(int ch);    // Returns 1 if ERR
int     kv_dma_ch_wait(int ch);     // Block until complete; returns 0 or -1 (error)
```

8.6.2. High-Level Transfer Functions

```
// 1D linear copy: resets ch, programs registers, starts, waits; returns 0 or -1
int kv_dma_1d_copy(int ch, uint32_t src, uint32_t dst, uint32_t bytes);

// 2D strided copy
int kv_dma_2d_copy(int ch, uint32_t src, uint32_t dst,
                   uint32_t row_bytes, uint32_t num_rows,
                   uint32_t src_stride, uint32_t dst_stride);
```

```
// 3D planar copy
int kv_dma_3d_copy(int ch, uint32_t src, uint32_t dst,
                   uint32_t row_bytes, uint32_t num_rows, uint32_t num_planes,
                   uint32_t src_stride, uint32_t dst_stride,
                   uint32_t src_pstride, uint32_t dst_pstride);

// Scatter-gather from descriptor list
int kv_dma_sg_copy(int ch, uint32_t sg_addr, uint32_t sg_count);
```

All high-level functions call `kv_dma_ch_wait()` internally and return `0` on success or `-1` on AXI error. The caller must call `kv_dma_ch_reset()` afterwards to clear the DONE flag.

8.6.3. Interrupt Functions

```
void     kv_dma_enable_irq(int ch);          // Set IE in CTRL + global IRQ_EN bit
void     kv_dma_disable_irq(int ch);         // Clear IE in CTRL + global IRQ_EN bit
uint32_t kv_dma_get_irq_status(void);        // Read IRQ_STAT (one bit per channel)
void     kv_dma_clear_irq(int ch);           // W1C clear IRQ_STAT bit for channel
```

8.6.4. Performance Counter Functions

```
void     kv_dma_perf_enable(void);           // Enable cycle/byte counting
void     kv_dma_perf_disable(void);          // Stop counting
void     kv_dma_perf_reset(void);            // Reset counters and re-enable
uint32_t kv_dma_perf_get_cycles(void);       // Cycles elapsed
uint32_t kv_dma_perf_get_rd_bytes(void);     // Total bytes read
uint32_t kv_dma_perf_get_wr_bytes(void);     // Total bytes written
```

8.6.5. Identification and Capability

```
uint32_t kv_dma_get_id(void);                // Read ID register (0xD4A00100)
uint32_t kv_dma_get_capability(void);        // Full 32-bit capability register
uint32_t kv_dma_get_num_channels(void);      // Number of DMA channels [15:8]
uint32_t kv_dma_get_max_burst_len(void);     // Maximum AXI burst length [7:0]
uint32_t kv_dma_get_version(void);           // Hardware version [31:16]
```

```
#include "kv_cap.h"
uint32_t cap = kv_dma_get_capability();
printf("DMA cap: 0x%08X  ch=%u burst=%u\n",
       cap, kv_dma_get_num_channels(), kv_dma_get_max_burst_len());
```

8.7. Performance Monitoring

```c
#include "kv_dma.h"

// Reset and enable performance counters
kv_dma_perf_reset();

// Start a 1D copy (blocking)
kv_dma_1d_copy(0, src_addr, dst_addr, 4096);
kv_dma_ch_reset(0);

// Stop counting so results are stable
kv_dma_perf_disable();

// Read statistics
uint32_t cycles   = kv_dma_perf_get_cycles();
uint32_t rd_bytes = kv_dma_perf_get_rd_bytes();
uint32_t wr_bytes = kv_dma_perf_get_wr_bytes();

printf("DMA: %u cycles, %u rd, %u wr\n", cycles, rd_bytes, wr_bytes);
```

Chapter 9. Timer Driver API

9.1. Overview

The Timer (`axi_timer.sv`) provides 4 independent 32-bit up-counters with PWM output, prescaler, dual compare, and per-channel interrupt support.

Base Address

`0x2004_0000` (KV_TIMER_BASE)

PLIC Sources

6–9 (`KV_PLIC_SRC_TIMER0` through `KV_PLIC_SRC_TIMER3` — one per channel)

Header

`kv_timer.h`

9.2. Register Map

Each timer channel occupies 0x20 bytes starting at `BASE + channel * 0x20`.

Register	Offset	Access	Description
COUNT	0x00	R/W	Current counter value (resets on COMPARE2 match)
COMPARE1	0x04	R/W	Compare 1 threshold (interrupt / PWM rising edge)
COMPARE2	0x08	R/W	Compare 2 threshold (period end + reload)
CTRL	0x0C	R/W	Control register (see bit table below)

Global registers (shared across all channels):

Register	Offset	Access	Description
INT_STATUS	0x80	W1C	Interrupt status — one bit per channel (bits 3:0)
INT_ENABLE	0x84	R/W	Interrupt enable — one bit per channel (bits 3:0)
CAPABILITY	0x88	RO	`{VERSION[31:16], NUM_CHANNELS[15:8], COUNTER_WIDTH[7:0]}`

CTRL bit fields:

Bit	Name	Description
0	EN	Timer enable
1	PWM_EN	PWM mode enable
3	INT_EN	Interrupt enable
4	PWM_POL	PWM polarity (1 = active high)
[31:16]	PRESCALE	Clock prescaler (counts at clock / (PRESCALE + 1))

9.3. API Functions

9.3.1. Initialization

```
void kv_timer_init(void);                          // Disable all channels, clear
counters
void kv_timer_init_irq(uint32_t timer_mask);       // Enable per-channel global IRQ
(bits 3:0)
```

9.3.2. Timer Mode

```
// Start timer ch; period in ticks (counter resets at period-1); prescale divides
clock
void kv_timer_start(int ch, uint32_t period, uint16_t prescale);

// Start timer with dual compare (compare1 = mid-period, period = end)
void kv_timer_start_dual(int ch, uint32_t compare1, uint32_t period,
                         uint16_t prescale, int int_en);

void     kv_timer_stop(int ch);                    // Disable timer channel
uint32_t kv_timer_get_count(int ch);               // Read current counter
```

9.3.3. PWM Mode

```
// Start PWM: goes high at 0, low at duty, reloads at period-1
void kv_timer_pwm_start(int ch, uint32_t period, uint32_t duty, uint16_t prescale);

// Update duty cycle without restarting
void kv_timer_pwm_set_duty(int ch, uint32_t period, uint32_t duty);

void kv_timer_pwm_stop(int ch);                    // Stop PWM output
```

9.3.4. Interrupt Handling

```
uint32_t kv_timer_get_int_status(void);            // Read INT_STATUS (bits 3:0)
void     kv_timer_clear_int(uint32_t mask);        // W1C clear (pass bit mask)
int      kv_timer_is_int_pending(int ch);          // Returns 1 if ch interrupt
pending
```

9.3.5. Capability Register

```
uint32_t kv_timer_get_capability(void);            // Full 32-bit capability register
uint32_t kv_timer_get_num_channels(void);          // [15:8] — number of timer
```

```
channels
uint32_t kv_timer_get_counter_width(void);          // [7:0]  — counter width in bits
uint32_t kv_timer_get_version(void);                // [31:16] — hardware version
```

```
#include "kv_cap.h"
#include "kv_timer.h"
uint32_t cap = kv_timer_get_capability();
assert(cap == KV_CAP_TIMER);   // 0x00010420: v1, 4 channels, 32-bit
printf("Timer: ch=%u width=%u\\n",
       kv_timer_get_num_channels(),   // 4
       kv_timer_get_counter_width()); // 32
```

9.4. Usage Example

```
#include "kv_timer.h"
#include "kv_plic.h"

volatile int ticks = 0;

void plic_timer_isr(void) {
    ticks++;
    kv_timer_clear_int(1 << 0);   // Clear channel 0 interrupt
}

int main(void) {
    kv_timer_init();

    // Generate 1 kHz interrupt (50 MHz clock, prescale=0, period=50000)
    kv_timer_init_irq(0x1);       // Enable channel 0 global IRQ
    kv_timer_start(0, 50000, 0);  // period=50000 ticks

    kv_plic_set_priority(KV_PLIC_SRC_TIMER0, 1);
    kv_plic_enable_source(KV_PLIC_SRC_TIMER0);
    kv_irq_enable();

    while (ticks < 100) {}        // Wait for 100 ms
    kv_timer_stop(0);
    kv_magic_exit(0);
}
```

Chapter 10. GPIO Driver API

10.1. Overview

The GPIO controller (`axi_gpio.sv`) provides 4 independent 32-pin banks with configurable direction, atomic set/clear, edge/level interrupt detection, and per-bank loopback mode for testing.

Base Address

`0x2005_0000` (KV_GPIO_BASE)

PLIC Source

5 (KV_PLIC_SRC_GPIO)

Header

`kv_gpio.h`

10.2. Register Map

Register banks are organized per pin-bank (banks 0–3, 32 pins each).

Register	Bank-0 Offset	Access	Description
DATA_OUT	0x00–0x0C	R/W	Output data (bank 0–3 at +0,+4,+8,+C)
SET	0x10–0x1C	WO	Atomic set bits (write-1-to-set)
CLEAR	0x20–0x2C	WO	Atomic clear bits (write-1-to-clear)
DATA_IN	0x30–0x3C	RO	Input data (read current pin state)
DIR	0x40–0x4C	R/W	Direction (1=output, 0=input)
IE	0x50–0x5C	R/W	Interrupt enable mask
TRIGGER	0x60–0x6C	R/W	Trigger mode (1=edge, 0=level)
POLARITY	0x70–0x7C	R/W	Polarity (edge: 1=rising; level: 1=high)
IS	0x80–0x8C	W1C	Interrupt status
LOOPBACK	0x90–0x9C	R/W	Loopback enable (output → input path)

Register	Bank-0 Offset	Access	Description
CAPABILITY	0xA0	RO	{VERSION[31:16], NUM_BANKS[15:8], NUM_PINS[7:0]}

10.3. API Functions

10.3.1. Initialization

```
void kv_gpio_init(void);                          // All inputs, IRQ off, loopback
off
```

10.3.2. Direction and Output

```
void     kv_gpio_set_dir(uint32_t bank, uint32_t mask);   // Set direction bits
uint32_t kv_gpio_get_dir(uint32_t bank);                  // Read direction
void     kv_gpio_write(uint32_t bank, uint32_t value);    // Write output register
uint32_t kv_gpio_read_out(uint32_t bank);                 // Read output register
void     kv_gpio_set(uint32_t bank, uint32_t mask);       // Atomic set (W1S)
void     kv_gpio_clear(uint32_t bank, uint32_t mask);     // Atomic clear (W1C)
void     kv_gpio_toggle(uint32_t bank, uint32_t mask);    // Toggle output bits
```

10.3.3. Input

```
uint32_t kv_gpio_read(uint32_t bank);                     // Read current pin state
```

10.3.4. Interrupt Configuration

```
void     kv_gpio_set_ie(uint32_t bank, uint32_t mask);       // Enable IRQ on pins
uint32_t kv_gpio_read_ie(uint32_t bank);
void     kv_gpio_set_trigger(uint32_t bank, uint32_t mask);  // 1=edge, 0=level
uint32_t kv_gpio_read_trigger(uint32_t bank);
void     kv_gpio_set_polarity(uint32_t bank, uint32_t mask); // 1=rising/high,
0=falling/low
uint32_t kv_gpio_read_polarity(uint32_t bank);
```

10.3.5. Interrupt Status

```
uint32_t kv_gpio_get_is(uint32_t bank);                   // Read interrupt status
void     kv_gpio_clear_is(uint32_t bank, uint32_t mask);  // W1C clear IRQ bits
```

10.3.6. Loopback

```
void     kv_gpio_set_loopback(uint32_t bank, uint32_t mask);  // Route out→in
(testing)
uint32_t kv_gpio_read_loopback(uint32_t bank);
```

10.3.7. Capability Register

```
uint32_t kv_gpio_get_capability(void);  // Full 32-bit capability register
uint32_t kv_gpio_get_num_pins(void);    // [7:0]  — pins per bank
uint32_t kv_gpio_get_num_banks(void);   // [15:8] — number of banks
uint32_t kv_gpio_get_version(void);     // [31:16] — hardware version
```

```
#include "kv_cap.h"
#include "kv_gpio.h"
uint32_t cap = kv_gpio_get_capability();
assert(cap == KV_CAP_GPIO);   // 0x00010104: v1, 1 bank, 4 pins
printf("GPIO: banks=%u pins_per_bank=%u\\n",
       kv_gpio_get_num_banks(), kv_gpio_get_num_pins());
```

Chapter 11. Watchdog Timer (WDT) Driver API

11.1. Overview

The WDT (`axi_wdt.sv`) counts down from a software-configured LOAD value and, on expiry, either raises a PLIC interrupt (IRQ mode) or asserts the `wdt_reset_o` hardware reset output (reset mode).

Base Address

`0x2006_0000` (KV_WDT_BASE)

PLIC Source

10 (`KV_PLIC_SRC_WDT`) — IRQ mode only

Header

`kv_wdt.h`

11.2. Quick Reference

```
// Start WDT: set reload value and expiry action.
//   load    - countdown start value (cycles)
//   intr_en - 1 = IRQ on expiry (PLIC src 10); 0 = hardware reset
void kv_wdt_start(uint32_t load, int intr_en);

// Reload COUNT from LOAD ("pet" the watchdog).
void kv_wdt_kick(void);

// Disable the WDT (clear EN bit).
void kv_wdt_stop(void);

// Clear WDT_INT status bit (W1C).  Call from ISR before kv_plic_complete().
void kv_wdt_clear_int(void);
```

11.3. Usage — IRQ Mode

```
#include "kv_wdt.h"
#include "kv_plic.h"
#include "kv_irq.h"

static void wdt_handler(uint32_t cause) {
    (void)cause;
    uint32_t src = kv_plic_claim();
    if (src == KV_PLIC_SRC_WDT) {
        kv_wdt_clear_int();
        kv_wdt_kick();
```

```
    }
    kv_plic_complete(src);
}

void setup_wdt_irq(void) {
    kv_irq_register(KV_CAUSE_MEI, wdt_handler);
    kv_plic_init_source(KV_PLIC_SRC_WDT, 1);
    kv_irq_enable();
    kv_wdt_start(100000u, 1);   /* LOAD=100000, INTR_EN=1 */
}
```

11.4. Usage — Reset Mode

```
#include "kv_wdt.h"

void setup_wdt_reset(void) {
    /* Start in hardware-reset mode.  wdt_reset_o latches on expiry.
     * System integrator feeds wdt_reset_o into the board reset tree. */
    kv_wdt_start(200000u, 0);   /* LOAD=200000, INTR_EN=0 */
}

/* In periodic supervisor loop: */
void periodic_kick(void) {
    kv_wdt_kick();
}
```

Chapter 12. Platform-Level Interrupt Controller (PLIC)

12.1. Overview

The PLIC (`axi_plic.sv`) manages external interrupt sources from peripherals.

Base Address

`0x0C00_0000` (KV_PLIC_BASE)

12.2. Interrupt Sources

ID	Source	Description
1	UART	Serial port RX ready or TX empty
2	SPI	SPI RX ready or TX empty
3	I2C	I2C RX ready, TX empty, or STOP done
4	DMA	DMA channel done or error
5	GPIO	GPIO pin interrupt (any bank)
6	Timer0	Timer channel 0 compare match (`KV_PLIC_SRC_TIMER0`)
7	Timer1	Timer channel 1 compare match (`KV_PLIC_SRC_TIMER1`)
8	Timer2	Timer channel 2 compare match (`KV_PLIC_SRC_TIMER2`)
9	Timer3	Timer channel 3 compare match (`KV_PLIC_SRC_TIMER3`)

12.3. API Functions

12.3.1. Source Control

```
void kv_plic_set_priority(uint32_t src, uint32_t priority);
void kv_plic_enable_source(uint32_t src);
void kv_plic_disable_source(uint32_t src);
int kv_plic_is_pending(uint32_t src);
void kv_plic_set_threshold(uint32_t threshold);
```

Parameters:

- `src`: Source ID (1-31)
- `priority`: Priority level (0-7, where 0 = disabled, 7 = highest)
- `threshold`: Only sources with priority > threshold are forwarded

12.3.2. Interrupt Handling

```
uint32_t kv_plic_claim(void);
void kv_plic_complete(uint32_t src);
```

Description:

- `kv_plic_claim()`: Read the highest-priority pending interrupt source (returns 0 if none)
- `kv_plic_complete()`: Signal completion of interrupt handling

12.3.3. Initialization Helper

```
void kv_plic_init_source(uint32_t src, uint32_t priority);
```

Convenience function that sets priority, enables source, sets threshold to 0, and enables external interrupts in `mie`.

Example:

```
// Initialize PLIC for UART interrupts
kv_plic_init_source(KV_PLIC_SRC_UART, 5);  // Priority 5

// Enable UART RX interrupt
kv_uart_irq_enable(KV_UART_IE_RX_READY);

// Enable global interrupts
kv_irq_enable();

// Interrupt handler
void external_irq_handler(void) {
    uint32_t src = kv_plic_claim();

    if (src == KV_PLIC_SRC_UART) {
        uint32_t status = kv_uart_irq_status();
        if (status & KV_UART_IE_RX_READY) {
            char c = kv_uart_getc();
            // Process character
        }
    }

    kv_plic_complete(src);
}
```

Chapter 13. Core-Local Interrupt Controller (CLINT)

13.1. Overview

The CLINT (`axi_clint.sv`) provides timer and software interrupts.

Base Address

`0x0200_0000` (KV_CLINT_BASE)

13.2. Features

- 64-bit free-running timer (`mtime`)
- 64-bit timer compare register (`mtimecmp`)
- Machine software interrupt (MSIP)

13.3. API Functions

13.3.1. Timer Operations

```
uint64_t kv_clint_mtime(void);
void kv_clint_set_mtimecmp(uint64_t cmp);
void kv_clint_timer_disable(void);
void kv_clint_timer_set_rel(uint64_t ticks);
void kv_clint_timer_irq_enable(void);
void kv_clint_timer_irq_disable(void);
```

Description:

- `kv_clint_mtime()`: Read current timer value (safe against high-word rollover)
- `kv_clint_set_mtimecmp()`: Set compare value for timer interrupt
- `kv_clint_timer_set_rel()`: Schedule interrupt N ticks from now

Example:

```
// Schedule timer interrupt in 1000000 cycles (~10ms at 100 MHz)
kv_clint_timer_set_rel(1000000);
kv_clint_timer_irq_enable();
kv_irq_enable();

// Timer interrupt handler
void timer_irq_handler(void) {
    // Handle timer interrupt
```

```
    // Re-arm for next interrupt
    kv_clint_timer_set_rel(1000000);
}
```

13.3.2. Software Interrupts

```
void kv_clint_msip_set(void);
void kv_clint_msip_clear(void);
void kv_clint_msip_irq_enable(void);
void kv_clint_msip_irq_disable(void);
```

Software interrupts can be used for inter-processor communication (IPI) in multi-core systems or for self-interrupts.

Chapter 14. Interrupt Management API

14.1. Overview

The `kv_irq.h` module provides high-level interrupt management functions.

14.2. Interrupt Source Bits (mie/mip)

Constant	Bit	Description
`KV_IRQ_MSIE`	3	Machine software interrupt enable
`KV_IRQ_MTIE`	7	Machine timer interrupt enable
`KV_IRQ_MEIE`	11	Machine external interrupt enable (PLIC)

14.3. Interrupt Cause Codes (mcause)

Constant	Code	Description
`KV_CAUSE_MSI`	3	Machine software interrupt
`KV_CAUSE_MTI`	7	Machine timer interrupt
`KV_CAUSE_MEI`	11	Machine external interrupt

14.4. Exception Cause Codes

Constant	Code	Description
`KV_EXC_INSN_MISALIGN`	0	Instruction address misaligned
`KV_EXC_INSN_FAULT`	1	Instruction access fault
`KV_EXC_ILLEGAL_INSN`	2	Illegal instruction
`KV_EXC_BREAKPOINT`	3	Breakpoint (EBREAK)
`KV_EXC_LOAD_MISALIGN`	4	Load address misaligned
`KV_EXC_LOAD_FAULT`	5	Load access fault
`KV_EXC_STORE_MISALIGN`	6	Store address misaligned
`KV_EXC_STORE_FAULT`	7	Store access fault
`KV_EXC_ECALL_M`	11	Environment call from M-mode

14.5. API Functions

14.5.1. Global Interrupt Control

```
void kv_irq_enable(void);
```

```
uint32_t kv_irq_disable(void);
void kv_irq_restore(uint32_t saved_mstatus);
```

Example:

```
// Critical section
uint32_t mstatus = kv_irq_disable();
// ... critical code ...
kv_irq_restore(mstatus);
```

14.5.2. Per-Source Control

```
void kv_irq_source_enable(uint32_t mask);
void kv_irq_source_disable(uint32_t mask);
int kv_irq_source_is_enabled(uint32_t mask);
```

Example:

```
// Enable timer and external interrupts
kv_irq_source_enable(KV_IRQ_MTIE | KV_IRQ_MEIE);

// Disable timer interrupt
kv_irq_source_disable(KV_IRQ_MTIE);
```

14.5.3. Wait For Interrupt

```
void kv_wfi(void);
```

Suspends the core until any enabled interrupt fires (`wfi` instruction). Use inside idle loops to reduce active power consumption:

```
kv_irq_enable();
while (!done)
    kv_wfi();
```

14.5.4. Handler Registration

Interrupt and exception handlers are registered per cause code. Both table slots default to built-in no-op / diagnostic handlers.

```
// Interrupt handler: receives the stripped cause code (mcause MSB cleared)
typedef void (*kv_irq_handler_t)(uint32_t cause);
```

```
// Exception handler: receives a pointer to the full saved register frame
typedef void (*kv_exc_handler_t)(kv_trap_frame_t *frame);

void kv_irq_register(uint32_t cause, kv_irq_handler_t handler);
void kv_exc_register(uint32_t cause, kv_exc_handler_t handler);
```

- `kv_irq_register()` — binds `handler` to interrupt cause `cause` (e.g. `KV_CAUSE_MEI`).
- `kv_exc_register()` — binds `handler` to exception cause `cause` (e.g. `KV_EXC_ILLEGAL_INSN`).
- Pass `NULL` to restore the built-in default (prints diagnostics + halts for exceptions).

Example:

```
static void on_timer(uint32_t cause) {
    (void)cause;
    timer_ticks++;
    kv_clint_timer_set_rel(TICK_PERIOD);
}

static void on_illegal_insn(kv_trap_frame_t *frame) {
    printf("Illegal insn @ 0x%08x\n", frame->mepc);
    frame->mepc += 4;   // skip and resume
}

kv_irq_register(KV_CAUSE_MTI,        on_timer);
kv_exc_register(KV_EXC_ILLEGAL_INSN, on_illegal_insn);
kv_irq_enable();
```

Chapter 15. Exception Handling and Trap Frame

15.1. Overview

When a machine-mode trap fires the processor saves the machine state and jumps to `mtvec`, which points to the assembly stub `trap_vector` in `start.S`. That stub:

1. Allocates a 144-byte `kv_trap_frame_t` on the interrupted thread's stack.
2. Saves all 32 general-purpose registers and four CSRs (`mepc`, `mstatus`, `mcause`, `mtval`) into the frame.
3. Passes a **pointer to the frame** as the sole argument to `trap_handler()`.
4. On return, restores machine state from the frame, then executes `mret`.

Because the epilogue restores `mepc` from `frame→mepc`, exception handlers redirect the return PC by writing `frame→mepc` — no direct `csrw mepc` inside the handler.

15.2. Trap Frame Layout (`kv_trap_frame_t`)

Defined in `kv_irq.h`. Total size: 144 bytes, allocated on the interrupted thread's stack. The frame is the **authoritative** state used by `mret`.

Offset	Field	Register	Notes
+0	`_pad`	x0	Placeholder; x0 is hardwired zero
+4	`ra`	x1	Return address at trap entry
+8	`sp`	x2	Stack pointer at trap entry (post-allocation)
+12	`gp`	x3	Global pointer
+16	`tp`	x4	Thread pointer
+20	`t0`	x5	Temporary
+24	`t1`	x6	Temporary
+28	`t2`	x7	Temporary
+32	`s0`	x8 / fp	Saved register / frame pointer
+36	`s1`	x9	Saved register
+40	`a0`	x10	Argument / return value
+44	`a1`	x11	Argument / return value
+48	`a2`	x12	Argument
+52	`a3`	x13	Argument
+56	`a4`	x14	Argument
+60	`a5`	x15	Argument

Offset	Field	Register	Notes
+64	`a6`	x16	Argument
+68	`a7`	x17	Argument / syscall number (ecall convention)
+72	`s2–s11`	x18–x27	Saved registers (at offsets +72…+108)
+112	`t3–t6`	x28–x31	Temporaries (at offsets +112…+124)
+128	`mepc`	mepc CSR	**Writable** — write to redirect the `mret` return PC
+132	`mstatus`	mstatus CSR	Restored verbatim; read-only for normal handlers
+136	`mcause`	mcause CSR	Read-only; MSB set for interrupts, clear for exceptions
+140	`mtval`	mtval CSR	Read-only; faulting address (load/store) or bad instruction

The field names match the RISC-V ABI register names (`ra`, `sp`, `a0`…`a7`, `s0`…`s11`, `t0`…`t6`), so accessing a saved argument register in a handler is simply `frame→a0`.

15.3. Design Rationale — Frame Pointer Instead of Direct CSR Writes

15.3.1. Why the Old Approach Was Fragile

The previous interface passed `(mcause, mepc, mtval)` as plain scalars and expected exception handlers to call `write_csr_mepc(mepc + 4)` to skip a faulting instruction. That relied on the trap epilogue re-reading `mepc` from the live CSR just before `mret`. Two conditions break that assumption:

Nested interrupts

A higher-priority interrupt can preempt an executing handler and overwrite the `mepc` CSR with the preempted handler's PC. When the outer handler later calls `write_csr_mepc()`, its value is correct — but if the epilogue reads `mepc` from the frame (required by the nested-interrupt fix), that CSR write is ignored.

Epilogue restore ordering

The epilogue saves `mepc` in the frame and restores it from there to support nesting correctly. Any `write_csr_mepc()` call inside the handler is silently overwritten by `lw t0, 128(sp); csrw mepc, t0` on the way out. This was the root cause of the bus_err infinite-loop bug: the handler updated the CSR to skip the faulting instruction, but the epilogue restored the original (faulting) `mepc`, causing the same instruction to re-execute forever.

15.3.2. The Fix: Frame Is the Authoritative Source

The epilogue has one invariant: it reads `mepc` from `frame→mepc` and writes it to the CSR. Exception handlers that want to redirect execution write `frame→mepc`:

```
// WRONG — CSR write is overwritten by the epilogue's frame→CSR restore
static void bad_handler(kv_trap_frame_t *frame) {
    write_csr_mepc(frame->mepc + 4);   // ← clobbered on exit
}

// CORRECT — update the frame; epilogue copies frame->mepc to CSR before mret
static void good_handler(kv_trap_frame_t *frame) {
    frame->mepc += 4;                  // ← survives the epilogue
}
```

15.3.3. Additional Benefits

Single source of truth

What is in `frame→mepc` at handler return is exactly what `mret` uses. No reasoning about CSR side-effects or epilogue ordering required.

Debuggability

A panic handler or debugger sees the full machine state (all 32 GPRs plus four CSRs) behind one pointer. Stack traces can walk `frame→ra` and `frame→sp` without probing live hardware CSRs.

Register injection

An advanced handler (e.g. an `ecall` emulation layer) can write arbitrary values into `frame→a0` …`frame→a7` to synthesise return values, or rewrite the entire register context to emulate an instruction.

Nested-interrupt safety

Re-enabling interrupts inside a handler is safe regardless of nesting depth: each level has its own frame on the stack. Inner trap activity cannot corrupt the `mepc`/`mstatus` saved in outer frames.

15.3.4. Interaction with the RTOS Port

The mini-RTOS (`mrtos_riscv.c`) has its own `mrtos_trap_vector` with a 132-byte context frame optimised for fast task switching. Its `mrtos_trap_handler()` constructs a temporary `kv_trap_frame_t` on the C stack when forwarding to `kv_irq_dispatch()`, and calls `write_csr_mepc(frame.mepc)` to commit any exception-handler update before the RTOS epilogue restores from its own frame. This bridges the two layouts without merging them.

15.4. Writing Exception Handlers

Register with `kv_exc_register(cause, handler)`. The handler receives `kv_trap_frame_t *frame` and returns `void`. The epilogue restores machine state from the frame when the handler returns.

15.4.1. Skip a Faulting Instruction

```
static void bus_err_handler(kv_trap_frame_t *frame) {
    g_bus_err_caught = 1;
```

```
    g_bus_err_mcause = frame->mcause;  // KV_EXC_LOAD_FAULT or KV_EXC_STORE_FAULT

    /* Bits [1:0] != 0b11  →  16-bit RVC instruction, else 32-bit */
    uint16_t inst16 = *(volatile uint16_t *)frame->mepc;
    frame->mepc += ((inst16 & 0x3u) != 0x3u) ? 2u : 4u;
}

kv_exc_register(KV_EXC_LOAD_FAULT,  bus_err_handler);
kv_exc_register(KV_EXC_STORE_FAULT, bus_err_handler);
```

15.4.2. Redirect to a Pre-Stored Resume Address

When `mepc` itself is invalid (e.g. a null-function-call fault where `mepc = 0`), `mepc + N` is still unmapped. Store a safe resume address before the dangerous call:

```
static volatile uintptr_t g_resume;

static void insn_fault_handler(kv_trap_frame_t *frame) {
    g_caught = 1;
    frame->mepc = (uint32_t)g_resume;  // redirect to pre-stored address
}

kv_exc_register(KV_EXC_INSN_FAULT, insn_fault_handler);

// Capture the instruction immediately following the dangerous call:
uintptr_t resume_pc;
asm volatile ("la %0, 1f" : "=r"(resume_pc));
g_resume = resume_pc;
((void (*)(void))NULL)();   // call NULL → KV_EXC_INSN_FAULT, mepc = 0
asm volatile ("1:");         // handler redirects mret here
```

15.4.3. Emulate an ecall (Syscall ABI)

```
static void ecall_handler(kv_trap_frame_t *frame) {
    uint32_t nr = frame->a7;                 // syscall number in a7 (RISC-V
convention)
    frame->a0   = dispatch_syscall(nr,        // return value in a0
                                   frame->a0, frame->a1, frame->a2);
    frame->mepc += 4;                         // ecall is always 4 bytes
}

kv_exc_register(KV_EXC_ECALL_M, ecall_handler);
```

15.5. Writing Interrupt Handlers

Register with `kv_irq_register(cause, handler)`. The handler receives the stripped cause code

(`mcause & 0x7FFFFFFF`) and returns `void`. No frame modification needed.

```
static void mei_handler(uint32_t cause) {
    (void)cause;
    uint32_t src = kv_plic_claim();
    if (src == KV_PLIC_SRC_UART) {
        while (kv_uart_rx_ready())
            rx_buf[rx_head++] = kv_uart_getc();
    }
    kv_plic_complete(src);
}

kv_irq_register(KV_CAUSE_MEI, mei_handler);
kv_plic_init_source(KV_PLIC_SRC_UART, 1);
kv_irq_source_enable(KV_IRQ_MEIE);
kv_irq_enable();
```

15.6. Overriding `trap_handler` Entirely

`trap_handler()` is a `weak` symbol. Provide your own definition to replace the `kv_irq_register` / `kv_exc_register` dispatch table:

```
// my_test.c — replaces the default trap_handler for the linked binary
void trap_handler(kv_trap_frame_t *frame) {
    if (frame->mcause & 0x80000000u) {
        my_fast_irq_dispatch(frame);
    } else {
        printf("EXC cause=%u pc=0x%08x val=0x%08x\n",
               frame->mcause, frame->mepc, frame->mtval);
        frame->mepc += 4;   // skip and continue
    }
}
```

> Override `trap_handler` only when the registration table is too slow or inflexible (e.g. performance-critical bare-metal interrupt paths). For normal use, `kv_irq_register` and `kv_exc_register` compose well and leave room to add handlers later without rewriting existing code.

15.7. Trap Entry / Exit Flow

```
Trap fires (interrupt or exception)
     │
     ▯
trap_vector  (start.S, 4-byte aligned)
     │
     ├── addi sp, sp, -144          allocate kv_trap_frame_t on stack
```

```
     ├── sw x1..x31, 4..124(sp)     save all general-purpose registers
     ├── csrr+sw mepc   → [sp+128]  save mepc
     ├── csrr+sw mstatus→ [sp+132]  save mstatus
     ├── csrr+sw mcause → [sp+136]  save mcause
     ├── csrr+sw mtval  → [sp+140]  save mtval
     ├── mv a0, sp                  pass frame pointer as argument
     └── call trap_handler
                │
                ▼
           trap_handler()  [trap.c — weak symbol, override per binary]
                └── kv_irq_dispatch(frame)
                       │
                       ├── mcause[31]=1 → irq_table[code](code)
                       │                  interrupt handler; frame unchanged
                       └── mcause[31]=0 → exc_table[code](frame)
                                          frame->mepc writable to redirect mret
                │
                ▼  (return from trap_handler)
     ├── lw t0, 132(sp); csrw mstatus, t0  restore mstatus
     ├── lw t0, 128(sp); csrw mepc, t0     restore mepc ← from frame (authoritative)
     ├── lw x1..x31, 4..124(sp)            restore GPRs
     ├── addi sp, sp, 144                  free frame
     └── mret                              resume at frame->mepc
     │
    ▼
Interrupted code resumes (or redirected PC if handler wrote frame->mepc)
```

Chapter 16. Control and Status Registers (CSR)

16.1. Overview

The `csr.h` module provides inline functions for reading and writing RISC-V control and status registers.

16.2. Machine Information Registers

```
uint32_t read_csr_mvendorid(void);  // Vendor ID
uint32_t read_csr_marchid(void);    // Architecture ID
uint32_t read_csr_mimpid(void);     // Implementation ID
uint32_t read_csr_mhartid(void);    // Hardware thread ID
```

16.3. Machine Trap Setup

```
uint32_t read_csr_mstatus(void);
void write_csr_mstatus(uint32_t val);

uint32_t read_csr_misa(void);       // ISA and extensions

uint32_t read_csr_mie(void);
void write_csr_mie(uint32_t val);

uint32_t read_csr_mtvec(void);
void write_csr_mtvec(uint32_t val);
```

16.4. Machine Trap Handling

```
uint32_t read_csr_mscratch(void);
void write_csr_mscratch(uint32_t val);

uint32_t read_csr_mepc(void);       // Exception program counter
void write_csr_mepc(uint32_t val);

uint32_t read_csr_mcause(void);     // Exception cause
void write_csr_mcause(uint32_t val);

uint32_t read_csr_mtval(void);      // Trap value (faulting address)
void write_csr_mtval(uint32_t val);

uint32_t read_csr_mip(void);        // Interrupt pending
```

```
void write_csr_mip(uint32_t val);
```

16.5. Performance Counters

```
uint64_t read_csr_mcycle(void);     // Cycle counter
uint64_t read_csr_minstret(void);   // Instructions retired counter
```

Example:

```
// Measure execution time
uint64_t start = read_csr_mcycle();
// ... code to measure ...
uint64_t end = read_csr_mcycle();
uint64_t cycles = end - start;

// Measure instruction count
uint64_t start_insn = read_csr_minstret();
// ... code to measure ...
uint64_t end_insn = read_csr_minstret();
uint64_t instructions = end_insn - start_insn;

float cpi = (float)cycles / instructions;
```

Chapter 17. Magic Device (Simulation/Test)

17.1. Overview

The Magic Device provides a simple interface for firmware to communicate with simulation environments without requiring a full UART implementation.

Base Address

`0x4000_0000` (KV_MAGIC_BASE)

> This peripheral is only available in simulation (RTL testbench, kv32sim, Spike plugins). It will not be synthesized for production FPGA/ASIC.

17.2. Registers

Offset	Name	Description
`0x0000`	CONSOLE	Write character (low byte) to simulation console
`0x0004`	EXIT	Write exit code to signal simulation end
`0x1000-0x11FF`	NCM[127:0]	Non-cacheable memory window (`KV_NCM_BASE`, `KV_NCM_SIZE`) for uncached code/data tests

17.3. AXI Decode and Responses

The RTL `axi_magic` slave acknowledges only these decoded regions:

- `KV_MAGIC_CONSOLE_ADDR` (`0x4000_0000`)
- `KV_MAGIC_EXIT_ADDR` (`0x4000_0004`)
- `KV_NCM_BASE .. KV_NCM_BASE + KV_NCM_SIZE - 1` (`0x4000_1000-0x4000_11FF`)

Transactions outside these regions return AXI `SLVERR`.

NCM details:

- Capacity: 128 x 32-bit words (512 bytes)
- Word index: `addr[8:2]`
- Read: returns stored 32-bit word
- Write: stores full `wdata` word (no per-byte strobe merge in RTL)

17.4. Exit Code Encoding

The EXIT register uses HTIF tohost encoding:

- Success (code 0): Write `1`

- Failure (code N): Write `(N << 1) \| 1`

17.5. API Functions

```
void kv_magic_putc(char c);
void kv_magic_exit(int code);
```

Description:

- `kv_magic_putc()`: Output one character to simulation console
- `kv_magic_exit()`: Signal program exit and halt (infinite loop)

Related constants from `kv_platform.h`:

- `KV_MAGIC_BASE`, `KV_MAGIC_CONSOLE_ADDR`, `KV_MAGIC_EXIT_ADDR`
- `KV_NCM_BASE`, `KV_NCM_SIZE`

Example:

```
#include "kv_platform.h"

int main(void) {
    const char *msg = "Hello from KV32!\n";

    // Output string
    for (const char *p = msg; *p; p++) {
        kv_magic_putc(*p);
    }

    // Signal successful completion
    kv_magic_exit(0);  // Simulation will stop here

    return 0;
}
```

Chapter 18. Complete Examples

18.1. Example 1: UART Echo with Interrupts

```c
#include "kv_platform.h"
#include "kv_uart.h"
#include "kv_plic.h"
#include "kv_irq.h"

#define UART_BAUD_DIV 107  // 115200 baud at 100 MHz

volatile char rx_buffer[256];
volatile uint8_t rx_head = 0;
volatile uint8_t rx_tail = 0;

void external_irq_handler(void) {
    uint32_t src = kv_plic_claim();

    if (src == KV_PLIC_SRC_UART) {
        uint32_t status = kv_uart_irq_status();

        if (status & KV_UART_IE_RX_READY) {
            // Read all available characters
            while (kv_uart_rx_ready()) {
                rx_buffer[rx_head++] = kv_uart_getc();
            }
        }
    }

    kv_plic_complete(src);
}

int main(void) {
    // Initialize UART
    kv_uart_init(UART_BAUD_DIV);
    kv_uart_puts("UART Echo Demo\n");

    // Configure PLIC and enable interrupts
    kv_plic_init_source(KV_PLIC_SRC_UART, 5);
    kv_uart_irq_enable(KV_UART_IE_RX_READY);
    kv_irq_enable();

    // Main loop: echo received characters
    while (1) {
        if (rx_head != rx_tail) {
            char c = rx_buffer[rx_tail++];
            kv_uart_putc(c);
        }
    }
```

```
    return 0;
}
```

18.2. Example 2: SPI Flash Read

```
#include "kv_platform.h"
#include "kv_spi.h"

#define SPI_CLK_DIV 49  // 1 MHz at 100 MHz system clock
#define FLASH_CMD_READ 0x03

void spi_flash_read(uint32_t addr, uint8_t *buf, uint32_t len) {
    uint8_t cmd[4];
    cmd[0] = FLASH_CMD_READ;
    cmd[1] = (addr >> 16) & 0xFF;
    cmd[2] = (addr >> 8) & 0xFF;
    cmd[3] = addr & 0xFF;

    kv_spi_cs_select(0);
    kv_spi_transfer_buf(cmd, NULL, 4);
    kv_spi_read_buf(buf, len);
    kv_spi_cs_deselect();
}

int main(void) {
    uint8_t data[256];

    // Initialize SPI
    kv_spi_init(SPI_CLK_DIV, KV_SPI_MODE0);

    // Read 256 bytes from address 0x00010000
    spi_flash_read(0x00010000, data, 256);

    // Process data...

    kv_magic_exit(0);
    return 0;
}
```

18.3. Example 3: I2C EEPROM Access

```
#include "kv_platform.h"
#include "kv_i2c.h"

#define I2C_CLK_DIV 249  // 100 kHz at 100 MHz
#define EEPROM_ADDR 0x50
```

```c
int eeprom_write(uint16_t mem_addr, const uint8_t *data, uint8_t len) {
    uint8_t buf[66];  // addr(2) + data(64 max)

    if (len > 64) return -1;

    buf[0] = (mem_addr >> 8) & 0xFF;
    buf[1] = mem_addr & 0xFF;
    for (uint8_t i = 0; i < len; i++) {
        buf[i + 2] = data[i];
    }

    return kv_i2c_master_write(EEPROM_ADDR, buf, len + 2);
}

int eeprom_read(uint16_t mem_addr, uint8_t *data, uint8_t len) {
    uint8_t addr_buf[2];
    addr_buf[0] = (mem_addr >> 8) & 0xFF;
    addr_buf[1] = mem_addr & 0xFF;

    // Write address
    kv_i2c_start();
    if (kv_i2c_write_byte((EEPROM_ADDR << 1) | 0) < 0) {
        kv_i2c_stop();
        return -1;
    }
    kv_i2c_write_byte(addr_buf[0]);
    kv_i2c_write_byte(addr_buf[1]);

    // Read data
    kv_i2c_start();  // Repeated START
    if (kv_i2c_write_byte((EEPROM_ADDR << 1) | 1) < 0) {
        kv_i2c_stop();
        return -1;
    }
    for (uint8_t i = 0; i < len; i++) {
        data[i] = kv_i2c_read_byte(i < len - 1);
    }
    kv_i2c_stop();

    return 0;
}

int main(void) {
    uint8_t write_data[] = {0xAA, 0xBB, 0xCC, 0xDD};
    uint8_t read_data[4];

    kv_i2c_init(I2C_CLK_DIV);

    // Write to EEPROM
    if (eeprom_write(0x0100, write_data, 4) < 0) {
```

```
        kv_magic_exit(1);
    }

    // Small delay for write cycle
    for (volatile int i = 0; i < 10000; i++);

    // Read back
    if (eeprom_read(0x0100, read_data, 4) < 0) {
        kv_magic_exit(2);
    }

    // Verify
    for (int i = 0; i < 4; i++) {
        if (read_data[i] != write_data[i]) {
            kv_magic_exit(3);
        }
    }

    kv_magic_exit(0);
    return 0;
}
```

18.4. Example 4: DMA Memory Copy with Performance Monitoring

```
#include "kv_platform.h"
#include "kv_dma.h"

#define DMA_CHANNEL 0
#define DATA_SIZE 4096

__attribute__((aligned(4))) uint8_t src_buffer[DATA_SIZE];
__attribute__((aligned(4))) uint8_t dst_buffer[DATA_SIZE];

int main(void) {
    // Initialize source buffer
    for (int i = 0; i < DATA_SIZE; i++) {
        src_buffer[i] = i & 0xFF;
    }

    // Reset and enable performance counters
    kv_dma_perf_reset();

    // Perform DMA copy using SDK (blocking 1D transfer)
    int ret = kv_dma_1d_copy(DMA_CHANNEL,
                             (uint32_t)src_buffer,
                             (uint32_t)dst_buffer,
                             DATA_SIZE);
```

```
    kv_dma_ch_reset(DMA_CHANNEL);  // Clear DONE flag

    if (ret != 0) {
        kv_magic_exit(1);  // AXI error
    }

    // Stop counting so results are stable
    kv_dma_perf_disable();

    // Read performance counters
    uint32_t cycles   = kv_dma_perf_get_cycles();
    uint32_t rd_bytes = kv_dma_perf_get_rd_bytes();
    uint32_t wr_bytes = kv_dma_perf_get_wr_bytes();

    printf("DMA: %u cycles, %u rd, %u wr\n", cycles, rd_bytes, wr_bytes);

    // Verify data
    for (int i = 0; i < DATA_SIZE; i++) {
        if (dst_buffer[i] != src_buffer[i]) {
            kv_magic_exit(2);
        }
    }

    kv_magic_exit(0);
    return 0;
}
```

Chapter 19. Build System Integration

19.1. Makefile Configuration

The SDK headers are located in `sw/include/`. Add this directory to your include path:

```
CFLAGS = -march=rv32ima_zicsr -mabi=ilp32 -O2 -g
CFLAGS += -I$(PROJECT_ROOT)/sw/include
```

19.2. Linker Script

The standard linker script (`sw/common/link.ld`) defines:

- `.text` section at `0x80000000` (RAM)
- `.data` and `.bss` sections following `.text`
- Stack pointer initialized to end of RAM
- Vector table at the start of `.text`

19.3. Startup Code

The startup code (`sw/common/start.S`) provides:

- Reset vector and exception vector table
- Stack pointer initialization
- BSS clearing
- Data section initialization (if loaded from ROM)
- Jump to `main()`

19.4. Common Runtime

Common runtime files in `sw/common/`:

- `trap.c`: Default exception handler
- `syscall.c`: Minimal newlib syscall stubs
- `printf.c`, `puts.c`, `putc.c`: Basic I/O functions

Chapter 20. Appendix A: Register Quick Reference

20.1. UART Registers

Register	Offset	Description
DATA	0x00	TX/RX data
STATUS	0x04	TX_BUSY(0), RX_READY(2)
IE	0x08	Interrupt enable
IS	0x0C	Interrupt status (W1C)
LEVEL	0x10	Baud rate divisor
CTRL	0x14	LOOPBACK(0)
CAP	0x18	Capability (RO): `{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}`

20.2. SPI Registers

Register	Offset	Description
CTRL	0x00	EN(0), CPOL(1), CPHA(2), CS[7:4]
DIV	0x04	Clock divisor
TX	0x08	TX FIFO push (WO)
RX	0x0C	RX FIFO pop (RO)
STATUS	0x10	BUSY(0), TX_READY(1), RX_VALID(2)
IE	0x14	Interrupt enable
IS	0x18	Interrupt status (W1C)
CAP	0x1C	Capability (RO): `{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}`

20.3. I2C Registers

Register	Offset	Description
CTRL	0x00	EN(0), START(1), STOP(2), READ(3), NACK(4)
DIV	0x04	Clock divisor
TX	0x08	Transmit data (WO)
RX	0x0C	Receive data (RO)
STATUS	0x10	BUSY(0), TX_READY(1), RX_VALID(2), ACK_RECV(3)

Register	Offset	Description
IE	0x14	Interrupt enable
IS	0x18	Interrupt status (W1C)
CAP	0x1C	Capability (RO): `{VERSION[31:16], TX_DEPTH[15:8], RX_DEPTH[7:0]}`

20.4. DMA Channel Registers

Register	Offset	Description
CTRL	0x00	Control and mode selection
STAT	0x04	BUSY(0), DONE(1), ERR(2)
SRC	0x08	Source address
DST	0x0C	Destination address
XFER	0x10	Transfer size (bytes)
SSTRIDE	0x14	Source stride (2D/3D)
DSTRIDE	0x18	Destination stride (2D/3D)
ROWCNT	0x1C	Row count (2D/3D)
SPSTRIDE	0x20	Source plane stride (3D)
DPSTRIDE	0x24	Destination plane stride (3D)
PLANECNT	0x28	Plane count (3D)
SG_ADDR	0x2C	Scatter-gather descriptor list base address
SG_COUNT	0x30	Number of scatter-gather descriptors

20.5. Timer Registers

Per-channel registers (channel *n* at `BASE + n * 0x20`):

Register	Offset	Description
COUNT	0x00	Current counter value (up-counter)
COMPARE1	0x04	Compare 1: mid-period interrupt / PWM falling edge
COMPARE2	0x08	Compare 2: period end (auto-reload)
CTRL	0x0C	EN(0), PWM_EN(1), INT_EN(3), PWM_POL(4), PRESCALE[31:16]

Global registers:

Register	Offset	Description
INT_STATUS	0x80	Interrupt status W1C (bits 3:0 = channels 3:0)

Register	Offset	Description
INT_ENABLE	0x84	Interrupt enable (bits 3:0)
CAPABILITY	0x88	Capability (RO): `{VERSION[31:16], NUM_CHANNELS[15:8], COUNTER_WIDTH[7:0]}`

20.6. GPIO Registers

All registers are replicated per bank (bank *b* at offset `b * 4` within group):

Register	Base Offset	Description
DATA_OUT 0–3	0x00–0x0C	Output data
SET 0–3	0x10–0x1C	Atomic set (write-1-to-set)
CLEAR 0–3	0x20–0x2C	Atomic clear (write-1-to-clear)
DATA_IN 0–3	0x30–0x3C	Input pin state (RO)
DIR 0–3	0x40–0x4C	Direction (1=output, 0=input)
IE 0–3	0x50–0x5C	Interrupt enable
TRIGGER 0–3	0x60–0x6C	Trigger (1=edge, 0=level)
POLARITY 0–3	0x70–0x7C	Polarity (edge: 1=rising; level: 1=high)
IS 0–3	0x80–0x8C	Interrupt status (W1C)
LOOPBACK 0–3	0x90–0x9C	Loopback mode (output → input)
CAPABILITY	0xA0	Capability (RO): `{VERSION[31:16], NUM_BANKS[15:8], NUM_PINS[7:0]}`

Chapter 21. Appendix B: Interrupt Vector Table

The default trap handler (`sw/common/trap.c`) dispatches interrupts and exceptions:

Code	Type	Handler
3	Machine Software Interrupt	Software interrupt handler
7	Machine Timer Interrupt	Timer interrupt handler
11	Machine External Interrupt	PLIC claim/complete sequence
0	Instruction Misaligned	Exception handler
2	Illegal Instruction	Exception handler
11	ECALL	System call handler

Chapter 22. Appendix C: Performance Considerations

22.1. DMA vs CPU Copy

For bulk data transfers > 256 bytes, DMA provides significant performance benefits:

- **CPU Copy**: ~4-8 cycles per word (load + store + loop overhead)
- **DMA**: 1-2 cycles per word (overlapped read/write, no CPU involvement)

22.2. Cache Coherency

The KV32 SoC includes both an instruction cache (I-cache) and a data cache (D-cache). When using DMA, explicit cache maintenance is necessary to keep the caches coherent with memory:

- **Before a DMA read** (CPU writes data, DMA reads): flush the D-cache range so dirty lines are written back to SRAM before the DMA transfer begins.

  ```
  kv_dcache_flush_range(buf, len);   // write-back dirty lines
  // then program the DMA descriptor and start the transfer
  ```

- **After a DMA write** (DMA writes data, CPU reads): invalidate the D-cache range so the CPU fetches fresh data from SRAM rather than stale lines.

  ```
  // wait for DMA transfer complete interrupt/poll
  kv_dcache_inval_range(buf, len);   // discard stale cached lines
  // now safe to read buf[]
  ```

- **Code loaded into SRAM by DMA** (e.g., overlay / firmware update): after invalidating the D-cache, also invalidate the I-cache so the CPU fetches the new instructions.

  ```
  kv_dcache_inval_range(buf, len);
  kv_fence_i();                      // synchronise I-cache
  ```

- Use `fence rw,rw` (or the `kv_fence()` helper) after MMIO writes to ensure store ordering before the DMA descriptor is submitted.

22.3. Interrupt Latency

Typical interrupt latency (cycles from assertion to handler entry):

- Minimum: ~15 cycles (no contention)
- Average: ~25 cycles (with PLIC arbitration)

- Maximum: ~100 cycles (worst-case pipeline state)

Chapter 23. Appendix D: Debugging Tips

23.1. Enable Instruction Trace

```
make rtl-mytest TRACE=1
```

Generates `rtl_trace.txt` with PC, instruction, and register values.

23.2. Generate Waveforms

```
make rtl-mytest WAVE=fst
```

Creates `wave.fst` for viewing in GTKWave.

23.3. Common Issues

No output from UART

Check baud rate divisor calculation

DMA transfer fails

Verify source and destination addresses are within valid memory ranges

Interrupts not firing

Ensure global interrupts enabled (`kv_irq_enable()`)

I2C NACK errors

Check device address and pull-up resistors on SCL/SDA

Chapter 24. Appendix E: RISC-V Compliance and KV32 Extensions

24.1. Scope

This appendix classifies SDK-visible features into three categories:

- Standard RISC-V ISA/privileged features
- Standard optional RISC-V extensions
- KV32-specific platform or architectural extensions (non-standard)

Use this section when evaluating portability to other RISC-V implementations.

24.2. Classification Matrix

Feature	Category	Notes / Portability Impact
RV32I base ISA	Standard	Fully standard base integer ISA behavior.
M extension (multiply/divide)	Standard optional	Standard extension; portable to cores that implement `M`.
A extension (atomics)	Standard optional	Standard extension; portable to cores that implement `A`.
Zicsr CSR access	Standard optional	Standard extension used by `csr.h` and interrupt/exception control flows.
Zifencei (`fence.i`)	Standard optional	Standard instruction. KV32 may add implementation-specific cache side effects (see note below).
Zicbom (`cbo.inval/clean/flush`)	Standard optional	Standard CMO instruction family when supported by the target. Software must feature-gate for cores without Zicbom.
PLIC programming model	Standard platform block	Register model follows standard RISC-V PLIC style; source IDs are SoC-specific.
CLINT programming model	Standard platform block	`mtime`, `mtimecmp`, and `msip` follow standard CLINT style.
PMA custom CSRs (`pmacfg*`, `pmaaddr*` at `0x7C0-0x7CB`)	KV32-specific (non-standard)	Runtime PMA region programming API in `kv_pma.h`. Not part of standard privileged CSR map.
Trap-frame API (`kv_trap_frame_t`, frame-pointer exception handlers)	KV32-specific SDK ABI	Design choice for robust trap handling; not required by standard RISC-V ABI.

Feature	Category	Notes / Portability Impact
Magic Device MMIO (`KV_MAGIC_BASE`)	KV32-specific simulation interface	Simulation/testbench-only host console+exit transport. Not synthesizable as a standard RISC-V device.
DMA / Timer / WDT / GPIO / UART / SPI / I2C register maps	KV32 platform-specific peripherals	Device APIs are KV32 SoC specific even when functionally similar to common IP blocks.

24.3. Important KV32 Notes

24.3.1. `fence.i` behavior

`kv_fence_i()` uses the standard `fence.i` instruction encoding, but KV32 documentation notes an additional implementation behavior: a full D-cache write-back may be performed to ensure self-modifying code visibility. Treat this as KV32-specific micro-architectural behavior, not a guaranteed generic RISC-V property.

24.3.2. PMA CSRs are custom

`kv_pma.h` exposes PMA control through custom M-mode CSRs in the `0x7C0-0x7CB` range. These CSRs are not part of the standard RISC-V privileged spec and must be abstracted out when porting firmware.

24.3.3. Trap redirection model

Exception handlers should update `frame→mepc` in `kv_trap_frame_t` rather than directly writing `mepc` CSR. This is the KV32 SDK contract and ensures compatibility with the trap epilogue restore sequence.

24.4. Portability Checklist

When porting KV32 firmware to another RISC-V target:

1. Replace or abstract `kv_pma_*` APIs.
2. Replace Magic Device calls (`kv_magic_putc`, `kv_magic_exit`) with target-specific transport.
3. Validate availability of `Zicbom` and `Zifencei` before using cache maintenance helpers.
4. Treat trap frame layout and handler ABI as platform-specific unless explicitly reproduced.
5. Re-map all peripheral drivers to target MMIO register maps and interrupt IDs.

Document Information

Version: 1.0

Last Updated: 2026-02-28

Copyright: © 2026 KV32 SoC Project

License: This documentation is provided under the MIT License.

For the latest version of this document and SDK updates, visit the project repository.